

Search Engine for Nested Documents

1. Problem Statement

Design a search engine that operates over **nested documents** (documents can reference other documents).

The system should:

- Return documents based on closest matching context
 - Display relevant matching lines/content
 - On click:
 - Navigate to the relevant part of the document
 - Load the full document
-

2. Goals

- Return document names as hyperlinks
- Display **2 lines (~25 words)** that best match the query
- Highlight query keywords

On Click

- Navigate to the **first matching paragraph**
 - Load the **full document**
-

3. System Design Overview

The system consists of three independent pipelines:

1. Crawling
 2. Indexing
 3. Serving
-

4. Functional Requirements

4.1 Crawling Pipeline

Document Structure

- Documents form a **graph**
 - Cycles can exist
 - Each document:
 - Has parent-child hierarchy
 - A child can have multiple parents
 - Size must be **< 10 MB**
-

Discovery Mechanism

Documents are discovered via:

- Hyperlinks within documents
 - User-submitted documents via registry
-

Traversal

- Hierarchical traversal
 - Maintain **visited set (docURL)** to avoid infinite loops
-

Event-Driven Crawling (Input to System)

Event Schema

```
{
  "eventId": "1234$20i023",
  "eventType": "NEW | UPDATE",
  "docURL": "http://abc.com",
  "timestamp": 17356490
}
```

```
}
```

Event Types

- 1. New Registry
- 2. High Priority Crawl
- 3. Subscription

Event Guarantees

- Idempotent
- Stored in **EventAudit DB**
- Deduplication using (eventId + docURL)
- TTL = 1 week

Document Prioritization

1. Subscription-Based Documents (Highest Priority)

- Real-time updates via events
- FIFO processing

2. High-Traffic Documents

- Based on Query Service traffic
- Crawled every hour
- Stored in **Traffic DB**

3. Registry-Based Documents (Lowest Priority)

- New/unclassified documents

Queue & Worker Model

Category	Queue	Worker
Subscription	Dedicated	Serverless

High Traffic	Dedicated	Always-on
Registry	Dedicated	Serverless

Document Storage (Crawler Output)

Temp Record (Initial Write)

```
{
  "docId": "String",
  "docUrl": "http://aac.com",
  "text": "Distributed systems are the hardest...",
  "timestamp": "epoch_millis",
  "stage": "temp"
}
```

Published Record (After SimHash)

```
{
  "docId": "String",
  "docUrl": "http://aac.com",
  "text": "Distributed systems are the hardest...",
  "timestamp": "epoch_millis",
  "stage": "publish",
  "simHash": "179$58"
}
```

Deduplication (SimHash)

NEW

- Compute SimHash

Distance	Action
≤ 8 bits	Duplicate
> 10 bits	New document

UPDATE

- Always processed
 - SimHash updated
-

Duplicate Handling

- Keep earliest document
 - Remove duplicates from search
 - Skip nested crawling for duplicates
-

Storage Lifecycle

1. Store as **temp**
 2. Generate SimHash
 3. Deduplicate
 4. Promote → **publish**
 5. Delete temp record
-

Parsing & Fragmentation

Documents are split into **paragraph-level fragments**

Fragment Schema

```
{
  "docId": "A",
  "fragments": [1, 2, 3],
  "byteOffsets": [0, 512, 1024]
}
```

Notes

- In our system, we avoid renumbering fragments on updates by using **gap-based sequencing** for **fragId**. Instead of assigning consecutive IDs like 1, 2, 3, we allocate IDs with gaps (e.g., 100, 200, 300). When a new fragment needs to be inserted between two existing fragments, we simply assign a value within the gap—for example, inserting between 100 and 200 results in 150. This allows us to maintain ordering without rewriting existing fragments or reindexing large

portions of the document. For instance, a document with fragments [100, 200, 300] can evolve to [100, 150, 200, 300] after an insertion, or [100, 200, 250, 300] with multiple updates. Only when gaps are exhausted (after many inserts in the same region) do we perform a **localized rebalancing** (e.g., renumber to 100, 110, 120, 130). This approach ensures efficient updates, minimizes write amplification, and supports fast range queries for fetching neighboring fragments.

Tokenization (Index Input)

```
{
  "docId": "String",
  "fragmentId": "String",
  "tokens": ["distributed", "systems", "hard"],
  "timestamp": "epoch_millis"
}
```

4.2 Indexing Pipeline

Purpose

Index documents for efficient retrieval

Consistency Model

- Eventual consistency
 - Updates triggered by:
 - NEW
 - UPDATE
 - DELETE
 - Runs every **5 seconds**
-

Index Update Rules

Scenario	Action
New word	Create entry
Existing word	Add doc reference
Word removed	Remove reference

Crawler → Indexing API

POST /send/index

```
{
  "requestId": "unique",
  "payload": {
    "docId": "String",
    "fragmentId": "frag_5",
    "timestamp": "epoch"
  }
}
```

Indexing → Callback

POST /callback/index/status

```
{
  "requestId": "String",
  "payload": {
    "status": "COMPLETED | RECEIVED | NOT_FOUND",
    "timestamp": "epoch"
  }
}
```

Processing Logic

if event.timestamp < current_doc_timestamp → skip

Inverted Index Schema

```
{
  "word": "indexing",
  "shardId": 4,
  "segmentId": "2023_A",
  "postings": [
    {
      "d": 1024,
      "f": 2,
      "p": [12, 45]
    }
  ],
  "max_ts": 1712345678000
}
```

Scaling Strategy

- Start with **10 shards**
- Scale per **1M documents**
- Partition:

$\text{hash}(\text{word}) \% N$

Update Handling (Tombstones)

Tombstone Storage Split

A. Persistent Layer (Cold)

```
{
  "tombstone_key": "docId_fragmentId",
  "deleted_at": 1712345679000
}
```

B. Query-Time Layer (Hot - In Memory)

Delete Bitset per Segment

Segment: 2023_A

Postings Index:

```
[0] → doc1_frag1  
[1] → doc1_frag2  
[2] → doc2_frag1
```

Delete Bitset:
[0, 1, 0]

Query Execution Flow

Search → Index Segment → iterate postings

```
for each posting i:  
  if bitset[i] == 1 → skip  
  else → include
```

Why this works

- O(1) check (RAM)
 - No DB lookup
 - Cache-friendly
 - Industry standard (Lucene-style)
-
- In our system, when a fragment is updated or deleted, the Crawler/Indexer writes a **tombstone record** (e.g., {docId, fragId}) into the Tombstone DB (persistent storage). The Indexing Service then maps this deletion to the corresponding **posting position** inside an index segment and updates the **in-memory delete bitset** maintained by the Search/Serving nodes. For example, suppose the inverted index (stored on disk) for the word “indexing” has postings [(doc1, frag1), (doc2, frag3), (doc3, frag2)]. If (doc1, frag1) is deleted, a tombstone is stored in the DB, and the search node updates its in-memory bitset to [1, 0, 0]. During query execution, the Serving layer reads postings from the index (disk/memory-mapped) and uses the bitset (RAM) to filter results: entries with bit 1 are skipped, and 0 are returned. This ensures that deletion handling is fast (in-memory) while the source of truth remains durable (Tombstone DB).

Failure Handling

- Idempotent indexing
- Resume from failure

- Master monitors workers
 - Restart safely
-

4.3 Serving Pipeline

Query Processing

- Remove stop words

Example:

"How can I learn python?" → "learn python"

Search Approach

- Semantic search
 - Ranking (TBD)
-

Retrieval

- Use inverted index
 - Fetch relevant fragments
-

Result Construction

Return:

- Ranked documents
- Top snippets (2 lines)

Techniques:

- Featured snippets
 - Passage ranking
-

Highlighting

- Highlight query keywords
-

On Click

- Load full document
 - Navigate to first matching fragment
-

Failure Handling

- Return error only
 - No partial results
-

Flow :

```
@startuml
title Search Engine - System Design

actor User

User --> QueryService : Search Query
QueryService --> QueryProcessor : preprocess
QueryProcessor --> IndexRouter : route

IndexRouter --> InvertedIndex : fetch postings
InvertedIndex --> QueryProcessor : postings

QueryProcessor --> RankingService : rank
RankingService --> SnippetService : select snippet

SnippetService --> FragmentStore : get fragment
FragmentStore --> SnippetService : fragment text

SnippetService --> QueryService : results
QueryService --> User : Top K Results

EventQueue --> CrawlerService : NEW / UPDATE
```

```
CrawlerService --> DocumentFetcher : fetch doc
DocumentFetcher --> Parser : extract text
Parser --> Fragmenter : split

Fragmenter --> SimHashService : compute hash
SimHashService --> SimHashDB : check/store

SimHashService --> DocumentDB : store doc
Fragmenter --> FragmentStore : store fragments

CrawlerService --> IndexingService : trigger indexing

IndexingService --> Tokenizer : tokenize
Tokenizer --> InvertedIndex : update index

IndexingService --> TombstoneDB : write deletes

@enduml
```

Data Model :

```
@startuml
title Search Engine - Data Model

entity "Document DB" as DocDB {
    +docId : String <<PK>>
    docUrl : String
    text : Text
    timestamp : long
    stage : temp | publish
    simHash : String
}

entity "Fragment Store" as FragDB {
    +fragmentId : String <<PK>>
    docId : String <<FK>>
    fragmentText : Text
    position : long
    timestamp : long
}

entity "Inverted Index" as IndexDB {
    +word : String
    shardId : int
    segmentId : String
}
```

```

    postings : List
    max_ts : long
}

entity "Postings" as Postings {
    d : int      ' docId
    f : int      ' fragmentId
    p : List     ' positions
}

entity "Tombstone DB" as TombDB {
    +tombstone_key : String <<PK>> ' docId_fragmentId
    deleted_at : long
}

entity "SimHash DB" as SimDB {
    +simHash : binary <<PK>>
    docId : String
    timestamp : long
}

entity "EventAudit DB" as EventDB {
    +eventId : String <<PK>>
    docUrl : String
    timestamp : long
    TTL : 1 week
}

entity "Traffic DB" as TrafficDB {
    +docId : String <<PK>>
    tier : HIGH | MED | LOW
    score : float
}

DocDB ||--o{ FragDB : "contains"
FragDB ||--o{ Postings : "indexed into"
IndexDB ||--o{ Postings : "stores"

DocDB ||--|| SimDB : "has hash"
FragDB ||--|| TombDB : "invalidated by"
EventDB ||--|| DocDB : "tracks"
TrafficDB ||--|| DocDB : "ranking signal"

@enduml

```

5. Non-Functional Requirements

Cost

- Minimize cost
-

Scalability

Phase	QPS
P0	10
P1	1M

Latency

Phase	Latency
P0	≤ 3 sec
P1	≤ 1 sec

6. Scope

P0

- 10 documents
 - Text only
 - Semantic search
-

P1

- PDF support
-

P2

- 1M documents
-

P3

- Web crawling
-

7. Additional Components

EventAudit DB

- Stores processed events
 - TTL = 1 week
 - Prevents duplicates
-

Traffic DB

- Stores document traffic classification
 - Updated daily
-